



ENSTA



IP PARIS



Projet Informatique Jeu 2D type Zelda



FISE 2028

10 juin 2026

Table des matières

1	Introduction	3
1.1	Problématique	3
1.2	Objectifs du projet	3
2	Analyse du problème	3
2.1	Définition exacte du projet	3
2.2	Choix technologiques et pistes envisagées	4
2.3	Hypothèses réductrices	4
2.4	Domaines d'utilisation	4
2.5	Limites observées	5
3	Architecture logicielle et description des classes	5
3.1	Vue d'ensemble	5
3.2	Diagramme de classes	6
3.3	Classes principales	6
3.4	Autres composants	9
4	Description des méthodes les plus importantes	11
4.1	Boucle de jeu – <code>GameScene.game_loop()</code>	11
4.2	Gestion des collisions – <code>is_blocking_rect()</code>	11
4.3	Pathfinding A*	11
4.4	Flood fill récursif	12
4.5	Système de combat	12
4.6	Transitions de salles – <code>TransitionManager</code>	12
5	Figures imposées	12
5.1	Détail des figures optionnelles	12
6	Tests et résultats	14
6.1	Tests unitaires	14
6.2	Benchmark A*	15
6.3	Tests manuels et débogage	15
7	Conclusion	15
7.1	Bilan des réalisations	16
7.2	Apports personnels	16
7.3	Perspectives et améliorations	16

1 Introduction

Dans le cadre de l'UE 2.1 « Projet Informatique », nous avons développé un jeu vidéo en deux dimensions en vue du dessus, inspiré de l'univers rétro du jeu vidéo *The Legend of Zelda* sur NES. Ce projet constitue une mise en pratique des notions de programmation orientée objet (POO) et de conception d'interfaces homme-machine (IHM) introduites en cours.

Lien vers le code du projet : https://github.com/Thr1lleX/Projet_Info

1.1 Problématique

La question centrale de ce projet est la suivante : *Comment développer un jeu vidéo d'action en temps réel fonctionnel en langage Python et quelles en sont les limites ?*

1.2 Objectifs du projet

Les objectifs que nous nous sommes fixés sont les suivants :

- approfondir notre maîtrise du langage Python ;
- mettre en œuvre les concepts fondamentaux de la programmation orientée objet (héritage, polymorphisme, encapsulation) ;
- comprendre et utiliser les mécanismes d'une IHM (boucle événementielle, gestion des entrées clavier, rendu graphique) ;
- concevoir un jeu 2D complet offrant une expérience de jeu cohérente, tout en laissant une part importante à notre créativité et à notre amour pour le jeu vidéo et la création de contenu multimédia (textures de jeu, musiques, sons, carte, histoire...).

2 Analyse du problème

2.1 Définition exacte du projet

Le projet consiste à développer un prototype de jeu d'action-aventure. Le joueur incarne un personnage évoluant au sein d'un monde ouvert composé de diverses salles et interconnectées et peuplé de créatures à combattre (et d'une histoire à finir). Son objectif est d'explorer les différents biomes, de survivre aux assauts des ennemis, de résoudre des énigmes et de progresser dans l'aventure (voire d'y trouver d'éventuels secrets laissés là par hasard?).

Le jeu intègre :

- un système de déplacement fluide avec gestion des collisions ;
- un bestiaire d'ennemis dotés d'une intelligence artificielle (IA) basée sur l'algorithme de recherche de chemin A* ;
- un système de combat avec plusieurs armes (épée, lance, boomerang, bombes) ;
- des boss à patterns de combat complexes ;
- une interface utilisateur (HUD¹) affichant les points de vie et l'inventaire ;
- un système de navigation entre différents écrans (titre, pause, inventaire, paramètres, fin de partie) ;

1. *Head-Up Display* : éléments d'information affichés en superposition sur l'écran de jeu (points de vie, inventaire, etc.).

- un système audio (musique d’ambiance et effets sonores);
- des transitions de salle animées;
- un système de sauvegarde et chargement de parties;
- des personnages non joueurs (PNJ) et un système de dialogues;
- des objets interactifs (coffres, portes verrouillées, interrupteurs, panneaux, points de sauvegarde);
- un monde composé de plus de 100 salles réparties en biomes distincts.

2.2 Choix technologiques et pistes envisagées

Framework graphique : PyQt5. Le choix de PyQt5 comme moteur graphique, imposé par l’encadrant, constitue une difficulté supplémentaire. En effet, PyQt5 n’est pas un framework spécialisé et optimisé pour la création de jeux vidéo, contrairement à des bibliothèques comme Pygame ou Pyglet. Concrètement, le jeu repose sur les classes `QGraphicsScene` et `QGraphicsView` de PyQt5, qui fournissent un moteur de rendu 2D basé sur une scène graphique. Le personnage, les ennemis et les éléments de décor sont tous des `QGraphicsPixmapItem` positionnés dans cette scène.

Audio : PyQtMultimedia & Pygame en complément. Avec l’accord de notre encadrant, nous avons utilisé le module `pygame.mixer` pour la gestion des effets sonores, PyQt5 présentant des limitations sur le nombre de canaux simultanés. La musique, quant à elle n’étant déclenchée (à quelques exceptions) qu’au chargement des salles, utilise `QSoundEffect` de PyQt.

Gestion des salles : JSON. Chaque salle du jeu est décrite dans un fichier JSON contenant la grille de tuiles (*tiles*), les positions d’apparition des ennemis et des interactifs, les transitions vers les salles adjacentes, le biome, les paramètres musicaux et les drapeaux conditionnels (*flags*). Ce format déclaratif offre une bonne lisibilité et facilite l’édition manuelle des niveaux.

2.3 Hypothèses réductrices

Afin de rendre le projet réalisable dans le temps imparti, nous avons adopté les simplifications suivantes :

- **Pas de moteur physique** : les déplacements et collisions sont gérés par des tests AABB (boîtes englobantes alignées sur les axes) sur une grille discrète, sans simulation de forces ni d’inertie;
- **Pas de réseau** : le jeu est strictement mono-joueur (*solo*);
- **Monde à grille fixe** : chaque salle est une grille de 16×11 tuiles, ce qui simplifie la gestion des collisions et du pathfinding en plus de simuler les jeux rétro du style;
- **Rendu synchrone** : toutes les mises à jour se font dans la boucle du `QTimer`, sans fil d’exécution (*thread*) séparé pour le rendu.

2.4 Domaines d’utilisation

Ce projet est essentiellement un prototype de jeu destiné au loisir. Il n’a ni l’ambition d’être commercialisé (problèmes de droit d’auteurs), ni celle d’être poursuivi sur du très long terme, au vu des limitations techniques inhérentes à PyQt5.

2.5 Limites observées

La principale limite que nous avons rencontrée concerne les performances de PyQt5 par rapport à des frameworks spécialisés :

- un plafond d'ennemis simultanés par salle, au-delà duquel le nombre d'appels à l'algorithme A* provoque des problèmes de performances;
- une gestion des sprites et des animations moins ergonomique que dans un moteur de jeu dédié;
- des ralentissements perceptibles lors de l'intégration de fonctionnalités audio avancées comme mentionné précédemment;

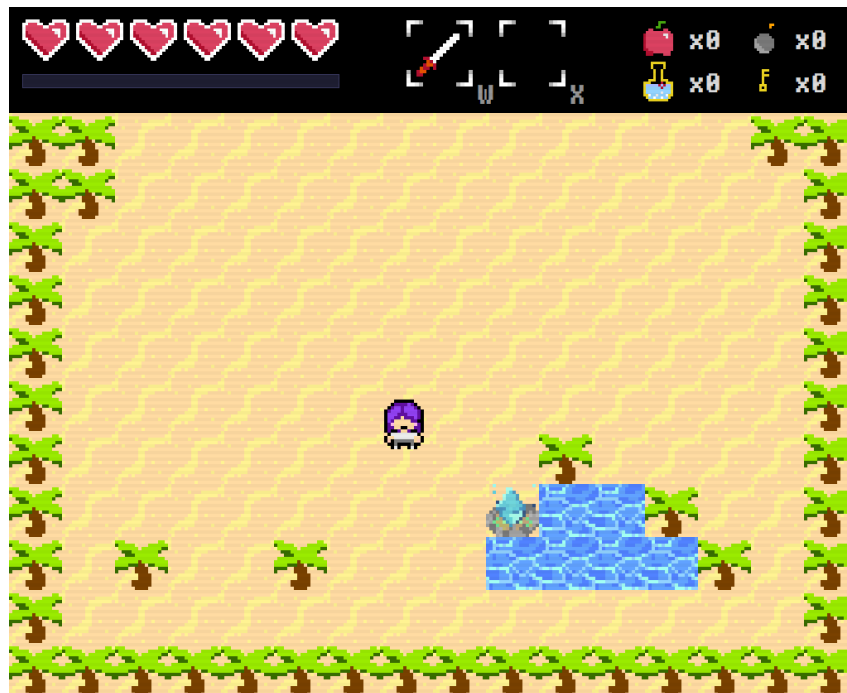


FIGURE 1 – Première salle du jeu

3 Architecture logicielle et description des classes

3.1 Vue d'ensemble

Le projet est organisé selon une architecture modulaire, répartie dans les répertoires suivants :

- `game/` – code source principal (entités, scène, écrans, etc.);
- `game/enemies/` – classes des ennemis (bestiaire);
- `game/attacks/` – classes des attaques (épée, lance, boomerang, bombes, boules de feu, etc.);
- `game/screens/` – écrans superposables (titre, pause, inventaire, sauvegarde, etc.);
- `game/interactables/` – objets interactifs (PNJ, coffres, portes, interrupteurs, panneaux);
- `game/ui/` – composants d'interface réutilisables;
- `assets/` – ressources graphiques (sprites, tuiles, HUD), organisées par biome;
- `rooms/` – fichiers JSON décrivant les salles;
- `mus/`, `sound_effect/` – ressources audio;

- `savefiles/` – fichiers de sauvegarde (JSON);
- `tests/` – tests unitaires et benchmarks.

Le point d'entrée du programme est le fichier `main.py`, qui instancie l'application PyQt5, crée la fenêtre de jeu (`GameWindow`), initialise le gestionnaire d'écrans (`ScreenManager`) et démarre la boucle événementielle.

3.2 Diagramme de classes

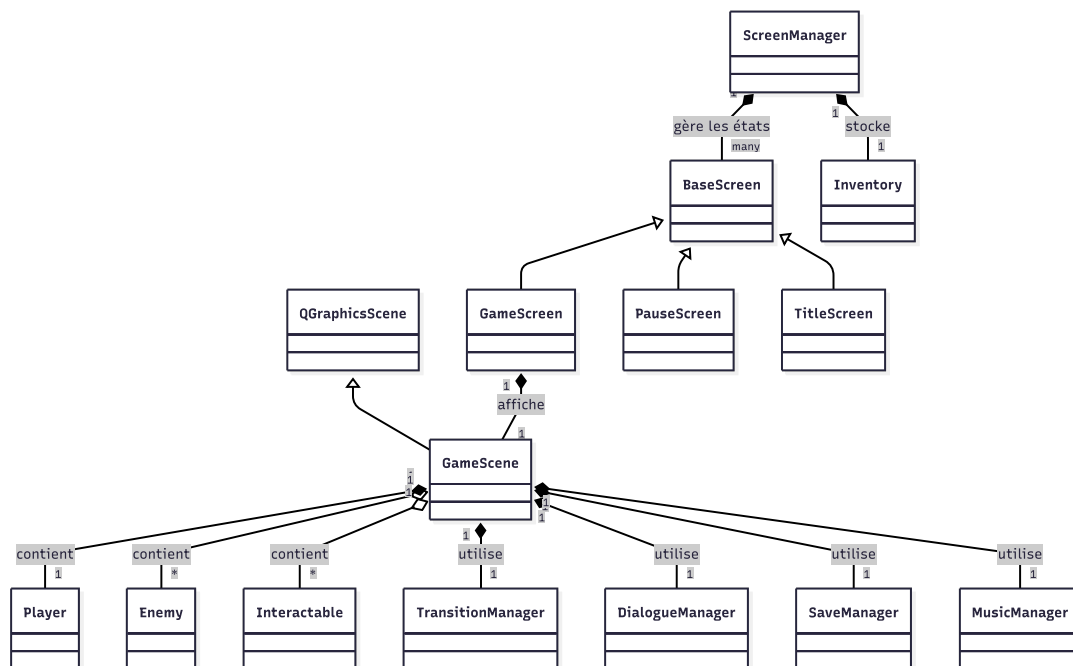


FIGURE 2 – Diagramme de classes UML du projet

3.3 Classes principales

3.3.1 Entity – Classe mère de toutes les entités

Une entité peut-être définie simplement comme un objet qui interagit avec l'environnement de jeu et le joueur lui-même. L'utilisateur incarne ainsi lui-même une entité "player". La classe `Entity` hérite de `QGraphicsPixmapItem` et constitue la brique fondamentale du système d'entités. Elle encapsule :

- la **position** de l'entité en coordonnées pixel flottantes (x, y);
- les **statistiques** de base : points de vie (pv_max, pv_main), vitesse ($speed$), défense;
- la **hitbox** : définie par un décalage et des dimensions, découplée du sprite pour plus de précision;
- les **mécanismes de combat** : prise de dégâts ($take_damage$), invulnérabilité temporaire, recul ($knockback$), étourdissement ($stun$), clignotement rouge et blanc.

La méthode abstraite `update(dt, scene)` impose à chaque sous-classe de définir son propre comportement à chaque *frame* (image affichée à l'écran). Le système de priorité est le suivant : $knockback > stun > \text{logique spécifique}$.

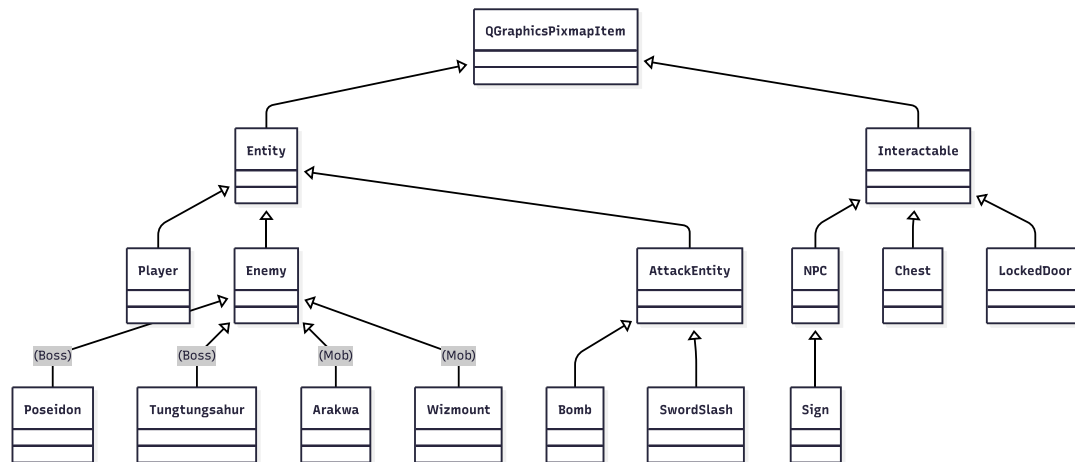


FIGURE 3 – Illustration de l'héritage de classes – Ici à 4 niveaux

3.3.2 Player – Le joueur

La classe Player hérite d'Entity et gère :

- les **entrées clavier** : un ensemble (set) de touches actives permet un déplacement fluide multi-directionnel avec normalisation en diagonale;
- les **attaques** : épée (SwordSlash), lance (Spear), boomerang (Boomerang), bombes (Bomb), magie de feu (Fireball), chacune instanciant une entité d'attaque indépendante;
- la **gestion de la mort** : appel à `scene.game_over()` lorsque les PV atteignent zéro;
- le **mécanisme de sortie** : maintien prolongé de la touche Échap pour quitter proprement.

3.3.3 GameScene – La scène de jeu

La classe GameScene hérite de QGraphicsScene et constitue le cœur du jeu. Elle est responsable de :

- la **boucle de jeu** : un QTimer cadencé à 60 FPS déclenche la méthode `game_loop()` qui met à jour toutes les entités;
- la **gestion des salles** : chargement depuis les fichiers JSON, affichage des tuiles, apparition des ennemis et des interactifs;
- la **détection des collisions** : vérification tuile par tuile via `is_blocking_rect()`;
- les **transitions** : délégation au TransitionManager pour les effets de fondu entre salles;
- l'**audio** : gestion de la musique de fond par salle via le MusicManager;
- le **système de drapeaux (flags)** : un mécanisme permettant de conditionner l'apparition d'ennemis, de PNJs et de transitions selon la progression du joueur.

3.3.4 Enemy – Classe de base des ennemis

La classe Enemy hérite d'Entity et ajoute :

- une **cible (target)**, généralement le joueur;
- une **portée d'aggro** : l'ennemi ne poursuit le joueur que s'il se trouve dans un rayon configurable, sinon il erre aléatoirement (wander);

- un **pathfinding A*** : délégation à la fonction `astar` du module `pathfinder` pour naviguer autour des obstacles;
- la **vérification de contact** : la méthode `try_hit_player()` détecte la collision AABB avec la cible et inflige dégâts, recul et étourdissement;
- un système de **butin** (*loot*) : chaque ennemi peut laisser tomber des objets à sa mort, avec une probabilité configurable.

Le bestiaire du jeu a été considérablement enrichi depuis notre rapport intermédiaire. Un registre dynamique (`enemy_registry.py`) scanne automatiquement le dossier `enemies/` et enregistre chaque classe disponible, permettant d'instancier les ennemis directement depuis les données JSON des salles. Parmi les ennemis les plus notables :

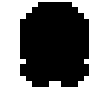
Nom du monstre	Spécificité	Sprite
Arakwa	Ennemi bondissant capable de sauter par-dessus les obstacles du décor pour surprendre le joueur.	
Wizmount	Mage redoutable attaquant à distance en lançant des boules de feu directionnelles fuyant si l'on s'approche.	
Shadow	Entité imitant et reproduisant les déplacements du joueur pour entraver ses mouvements ou le piéger.	
Tungtungsahur	Mini-boss de zone doté d'une machine à états alternant charges, attaques de zone et phases de vulnérabilité.	
Poseidon	Boss aquatique doté de patterns variés incluant des lancers de tridents différents et des phases de rage.	

TABLE 1 – Présentation des ennemis principaux du bestiaire

Ce bestiaire est encore amené à évoluer au fil du développement des biomes restants.

3.3.5 Interactables – Les objets interactifs

Le système d'objets interactifs repose sur une hiérarchie de classes héritant de la classe de base `Interactable`; elle-même héritant de `QGraphicsPixmapItem`, définissant une zone d'interaction et une méthode `interact()` virtuelle redéfinie par chaque sous-classe. Cette structure permet au joueur d'interagir uniformément avec l'environnement, comme détaillé dans le tableau 2 :

3.3.6 ScreenManager – Gestionnaire d'écrans

Le `ScreenManager` est le chef d'orchestre de la navigation entre les différents états de l'application : titre, jeu, pause, inventaire, fin de partie, paramètres, sélection de sauvegarde et contrôles. Il :

- maintient un dictionnaire d'écrans enregistrés (`BaseScreen`) et un état courant;
- route les événements clavier vers l'écran actif;
- gère la pause (gel du `QTimer`, baisse du volume);
- crée une `GameScene` fraîche lors d'un *reset* ou retour au menu;
- offre un point central pour les données persistantes (paramètres, inventaire).

Type d'objet	Spécificité	Sprite
NPC	Personnage non joueur pouvant déclencher des dialogues, avec gestion de dialogues conditionnels selon l'état des drapeaux.	
Sign	Panneau informatif, sous-classe de NPC, affichant des indications ou indices lors de l'interaction.	
Chest	Coffre contenant un objet, qui ne peut être ouvert qu'une seule fois (état persisté par un drapeau).	
LockedDoor	Porte verrouillée nécessitant une clé dans l'inventaire, qui modifie dynamiquement les tuiles de collision à l'ouverture (stocké également par un drapeau).	
CrystalSwitch	Interrupteur à cristal permettant de basculer l'état des blocs colorés de la salle (mécanisme d'énigmes).	
SavePoint	Point de sauvegarde permettant d'enregistrer la progression courante de la partie.	

TABLE 2 – Présentation des différents types d'objets interactifs

3.3.7 Pathfinder – Algorithme A* et flood fill

Le module `pathfinder.py` implémente l'algorithme de recherche de chemin A* adapté au contexte du jeu (IA basique). Cette fonctionnalité est essentielle car sans elle, une simple implémentation naïve permettant à l'ennemi de se diriger en ligne droite vers le joueur implique qu'il se trouve coincé dans certains lieux exigus de la grille (coins, cul-de-sac...). Le module `pathfinder.py` implémente les algorithmes de navigation des ennemis :

- **construction de la grille** : la fonction `get_walkable_grid()` convertit les données JSON de la salle en une grille booléenne (case libre / bloquante) ;
- **flood fill récursif** : la fonction `flood_fill()` effectue un parcours en profondeur (DFS) récursif pour déterminer l'ensemble des cases accessibles depuis un point donné, permettant de vérifier la connectivité avant un appel A* ;
- **recherche A*** : la fonction `astar()` utilise une file de priorité (heapq) avec une heuristique euclidienne pour trouver le chemin optimal ;
- **ligne de vue** : la fonction `line_of_sight()` effectue un *raycasting* par pas réguliers pour vérifier si un chemin direct existe, en tenant compte de la taille de l'entité ;
- **support des entités larges** : le pathfinding prend en compte la taille de la hitbox de l'ennemi pour éviter que les entités de grande taille se coincent dans les passages étroits.

3.4 Autres composants

Le tableau 3 récapitule les classes et modules secondaires du projet.

Module	Classe	Rôle
attacks/	SwordSlash	Attaque épée courte portée avec animation, hitbox rotative selon la direction.
	Spear	Lance longue portée (1 × 4 tuiles).
	Boomerang	Projectile à trajectoire aller-retour.
	Bomb / Explosion	Bombe posée au sol avec minuterie, explosant en zone circulaire (brisant les murs cassables).
	Fireball	Projectile de feu utilisé par le joueur et certains ennemis.
	Lightning	Projectile d'électricité utilisé par un ennemi aléatoirement sur les cases autour de lui.
screens/	BaseScreen	Classe mère de tous les écrans.
	TitleScreen	Écran titre avec menu.
	PauseScreen	Écran de pause superposé au jeu.
	InventoryScreen	Affichage et gestion de l'inventaire.
	SettingsScreen	Configuration (volume, zoom, contrôles).
	GameOverScreen	Écran de fin de partie.
	SaveSelectScreen	Sélection du slot de sauvegarde à charger.
	SaveMenuScreen	Sélection du slot de sauvegarde à sauvegarder.
	ControlsScreen	Affichage des contrôles.
music.py	MusicManager	Lecture et transitions musicales (fondu entrant/sortant) par salle.
sfx.py	SFXManager	Lecture des effets sonores.
transition.py	TransitionManager	Machine à états gérant les fondus entre salles.
hud.py	HUD	Affichage d'informations utiles (cœurs, slots d'items).
settings.py	Settings	Chargement et sauvegarde de paramètres modifiables dynamiquement (échelle du jeu, volume sonore...)
dialogue_m.	DialogueManager	Système de dialogues avec animation lettre par lettre et polices multiples.
save_m.	SaveManager	Gestion des sauvegardes (lecture/écriture JSON, drapeaux de progression).
item.py	Inventory	Grille de slots d'items avec sérialisation JSON et synchronisation avec les drapeaux.
config.py	–	Constantes globales (taille des tuiles, FPS, chemins).

TABLE 3 – Classes et modules secondaires du projet

4 Description des méthodes les plus importantes

Cette section détaille les algorithmes et mécanismes clés qui gouvernent le fonctionnement du jeu.

4.1 Boucle de jeu – `GameScene.game_loop()`

La boucle de jeu est cadencée par un `QTimer` à intervalle fixe de $\lfloor \frac{1000}{60} \rfloor = 16$ ms, soit environ 60 images par seconde (*FPS*). À chaque *tick*, la méthode `game_loop()` exécute dans l'ordre :

1. calcul du Δt réel (temps écoulé depuis le dernier appel);
2. mise à jour des effets visuels et des dialogues;
3. mise à jour des entités;
4. mise à jour des interactifs et ramassage des objets au sol;
5. mise à jour des transitions de salle et de la musique;
6. rafraîchissement du HUD.

L'utilisation d'un pas de temps fixe permet de conserver une simulation stable et reproductible, en évitant que de faibles variations de temps d'exécution n'affectent la vitesse des déplacements ou les interactions du jeu.

4.2 Gestion des collisions – `is_blocking_rect()`

La détection de collisions repose sur une vérification tuile par tuile. Pour un rectangle de hitbox donné (x, y, w, h) , la méthode identifie toutes les cases de la grille chevauchées et vérifie si l'une d'entre elles est bloquante. Les bords de la salle sont également considérés comme des obstacles (sauf dans la direction d'une transition définie).

Le déplacement d'une entité (méthode `Entity.move()`) sépare le mouvement en deux axes indépendants (X puis Y), permettant un « glissement » le long des murs plutôt qu'un arrêt complet. Un mécanisme de *correction de coin* (`corner_correction`) applique un léger décalage perpendiculaire pour permettre au joueur de contourner les angles sans se bloquer.

4.3 Pathfinding A*

L'algorithme A* est utilisé pour la navigation des ennemis. Son fonctionnement dans le contexte du jeu est le suivant :

1. la grille de la salle est convertie en graphe implicite (chaque case a 8 voisins potentiels);
2. l'heuristique euclidienne guide la recherche vers la cible;
3. lorsqu'une ligne de vue directe existe entre l'ennemi et le joueur, le pathfinding est court-circuité au profit d'un déplacement en ligne droite (optimisation);
4. le chemin est recalculé à intervalles réguliers (toutes les 50 ms) pour s'adapter aux déplacements du joueur;
5. le chemin brut est lissé par l'algorithme de *String Pulling* (`smooth_path`) qui supprime les points intermédiaires inutiles en vérifiant la ligne de vue.

La prise en compte de la taille de l'entité (`is_area_walkable()`) empêche les ennemis de taille supérieure à 1×1 tuile de s'engager dans des passages trop étroits.

4.4 Flood fill récursif

Le module `pathfinder.py` inclut une fonction récursive `flood_fill()` qui effectue un parcours en profondeur (DFS) sur la grille de la salle. À partir d'une case de départ, elle explore récursivement les quatre cases adjacentes (haut, bas, gauche, droite) et construit l'ensemble de toutes les cases atteignables.

Cette fonction est utilisée par `are_connected()` pour vérifier si deux positions sont dans la même zone accessible de la grille. Cela permet de court-circuiter un appel A* coûteux lorsque la cible est structurellement inatteignable (zones séparées par un mur complet).

4.5 Système de combat

Le système de combat repose sur plusieurs mécanismes coordonnés :

Prise de dégâts (`take_damage`). Lorsqu'une entité est touchée, la méthode retire les PV, déclenche un flash rouge (`apply_red_flash`), active une période d'invulnérabilité temporaire et calcule un recul (*knockback*) dirigé dans la direction opposée à la source de dégâts.

Knockback (`apply_knockback`). Le recul est appliqué progressivement à chaque *frame*, en respectant les collisions avec les murs. Si l'entité est bloquée par un obstacle, le recul s'arrête immédiatement.

Stun (`stun`). L'étourdissement bloque tous les déplacements et attaques de l'entité pendant une durée configurable. Un léger mouvement oscillatoire (*wiggle*) fournit un retour visuel au joueur.

4.6 Transitions de salles – `TransitionManager`

Le changement de salle suit une machine à états :

1. **fade_out** : assombrissement progressif de l'écran;
2. **change_room** : chargement de la nouvelle salle, nettoyage de la scène, repositionnement du joueur;
3. **fade_in** : éclaircissement progressif;
4. **music_wait** : attente optionnelle si la musique de la nouvelle salle diffère (transition musicale en fondu).

5 Figures imposées

Le tableau 4 recense les neuf figures imposées du projet (six communes et trois optionnelles) ainsi que les modules principaux dans lesquels elles sont implémentées.

5.1 Détail des figures optionnelles

Algorithme d'optimisation : A*. L'algorithme A* implémenté dans `pathfinder.py` est un algorithme d'optimisation de recherche de chemin. Il minimise une fonction de coût combinant la distance parcourue et une heuristique euclidienne. L'optimisation est renforcée par le lissage *String Pulling*

#	Figure	Réalisation	Modules clés
1	≥ 4 classes	Plus de 40 classes créées	entity.py, player.py, enemy.py, item.py, scene.py, etc.
2	Modules structurés	7 paquets, 40+ fichiers source	game/, enemies/, attacks/, screens/, interactables/
3	Héritage / Composition	Hierarchies à 4+ niveaux; composition dans GameScene	entity.py, attack_entity.py, interactable.py
4	Documentation	Docstrings et commentaires dans les fichiers clés	ensemble du projet
5	Tests unitaires	41 tests, 4 fichiers, ≥ 2 cas par méthode	tests/
6	Stockage données	Sauvegardes, paramètres et salles en JSON	save_manager.py, settings.py, rooms/
7	Algo. d'optimisation	A* avec lissage <i>String Pulling</i>	pathfinder.py
8	Design patterns	Machine à états, Registry, Template Method	transition.py, screen_manager.py, enemy_registry.py
9	Fonction récursive	flood_fill() – DFS sur la grille	pathfinder.py

TABLE 4 – Récapitulatif des neuf figures imposées

(smooth_path) et la vérification de ligne de vue (line_of_sight). Un benchmark (section 6) montre que chaque appel consomme de l'ordre de 5 % du budget *frame* à 60 FPS.

Design patterns. Plusieurs patrons de conception sont mis en œuvre dans le projet afin de garantir une architecture robuste :

- **Machine à états (State)** : le TransitionManager gère les phases de transition de salle (fade_out, change_room, fade_in, music_wait); le ScreenManager gère les modes de l'application; les boss Tungtungsahur et Poseidon utilisent des machines à états pour leurs patterns d'attaque;
- **Registry** : les modules enemy_registry.py scanne automatiquement son dossier cible et enregistre les classes disponibles dans un dictionnaire, permettant l'instanciation dynamique depuis les fichiers JSON;
- **Composite** : ce patron permet de traiter une collection d'objets disparates (entités, objets interactifs, attaques) de manière uniforme. La classe GameScene maintient ces éléments dans des structures distinctes mais délègue la mise à jour à l'écran en invoquant leur méthode update(dt, scene) commune, simplifiant ainsi le cycle de rafraîchissement;
- **Visiteur (Visitor)** : ce patron permet d'ajouter de nouvelles opérations sur des objets sans modifier leur classe. C'est le cas lors des interactions entre le joueur et les éléments du décor : la méthode interact(scene, player) de la classe Interactable prend le joueur en argument (self depuis la méthode appelante dans Player, via closest.interact(scene, self)), permettant à l'interactif d'exécuter des actions contextuelles dépendantes de l'état du

joueur sans couplage fort;

- **Memento** : utilisé pour capturer et externaliser l'état interne d'un objet (comme l'avancement dans GameScene et l'état du joueur) afin de pouvoir le restaurer ultérieurement. C'est la base de notre système de sauvegarde et de chargement (SaveManager) ainsi que du suivi de progression (get_flag et set_flag), assurant la persistance des données du jeu dans un fichier JSON externe sans violer l'encapsulation.

Fonction récursive : flood fill. La fonction `flood_fill()` (cf. section 4.4) effectue un parcours en profondeur récursif sur la grille de tuiles. Son intérêt dans le cœur du projet est de permettre une vérification rapide de connectivité : avant de lancer A^* , on peut vérifier que la cible est dans la même zone accessible, évitant ainsi un calcul inutile. Sur la grille $16 \times 11 = 176$ cases, la profondeur de récursion maximale reste largement en dessous de la limite Python par défaut (1000). Cette fonction récursive est au cœur du projet dans le sens où elle est appelée tout le temps et permet un gain de performances non négligeable si un ennemi ne peut pas atteindre le joueur.

Remarque : approche data-driven des fichiers JSON. Bien que nous ne la comptons pas parmi les trois figures optionnelles, notre format de description des salles en JSON s'approche d'un DSL (*Domain Specific Language*) dans la mesure où il définit une sémantique propre au domaine : grille de tuiles, transitions directionnelles, conditions d'apparition (`spawn_if`, `despawn_if`), drapeaux, biomes, paramètres musicaux. Cependant, nous reconnaissons que la syntaxe reste celle de JSON standard (pas de grammaire ni de parseur dédié), ce qui limite sa qualification stricte de DSL.

6 Tests et résultats

6.1 Tests unitaires

Nous avons développé 41 tests unitaires répartis en quatre fichiers, couvrant les modules pouvant être testés indépendamment de l'interface graphique (les modules PyQt5 sont simulés via `unittest.mock`) :

- `test_pathfinder.py` (14 tests) : heuristique, construction de la grille, A^* (chemin trouvé, grille bloquée, départ = arrivée), lissage de chemin, et flood fill récursif (grille ouverte, case bloquée, zones isolées);
- `test_inventory.py` (10 tests) : ajout d'items, dépassement de pile, item inconnu, comptage, consommation, équipement, sérialisation aller-retour, remise à zéro;
- `test_save_manager.py` (9 tests) : drapeaux (défaut, lecture, écriture, écrasement), santé, salle courante, persistance en fichier;
- `test_settings.py` (8 tests) : taille des tuiles selon le zoom, vitesse de base, chargement de valeurs, fichier manquant, sauvegarde, aller-retour sauvegarde/chargement.

L'ensemble des tests passe avec succès (`pytest tests/ -v : 41 passed`).

Benchmark A* — 200 appels par scenario Comparaison : A* seul vs are_connected + A*

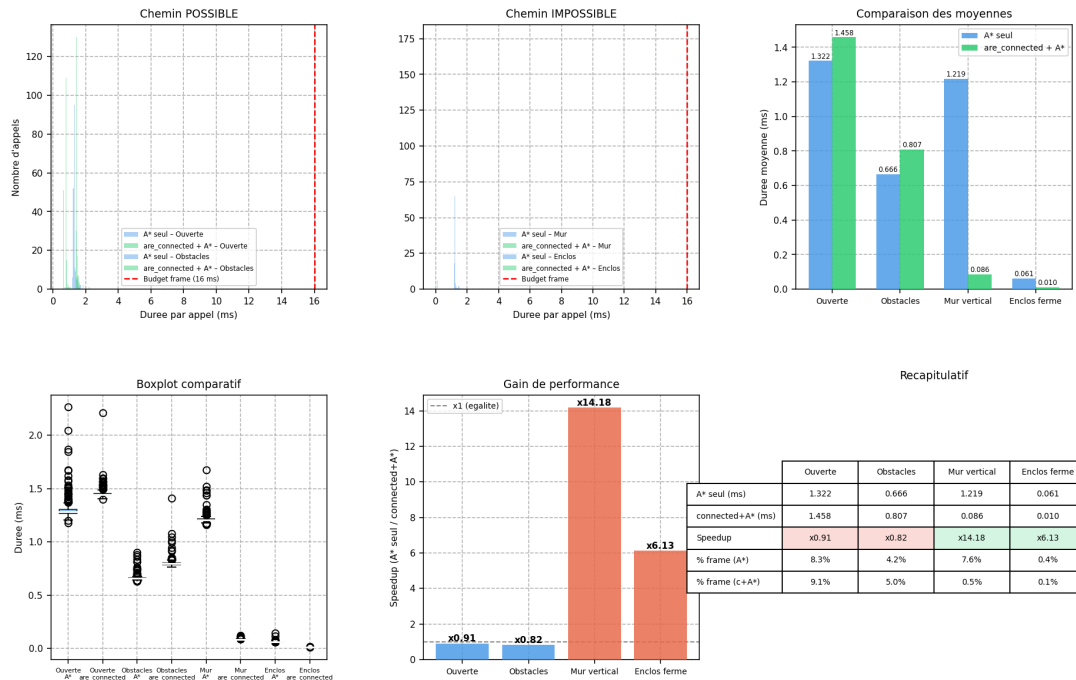


FIGURE 4 – Graphique du benchmark A*

6.2 Benchmark A*

Un script de benchmark (`tests/benchmark_astar.py`) mesure les performances de l'algorithme A* sur 200 appels dans deux scénarios : grille ouverte (0 % d'obstacles) et grille avec 25 % d'obstacles. Les résultats montrent que chaque appel consomme de l'ordre de 5 % du budget d'une *frame* à 60 FPS (env. 16 ms), validant l'utilisabilité en temps réel.

6.3 Tests manuels et débogage

Les tests manuels ont été effectués tout au long du développement dans le cadre du processus classique de débogage. Parmi les cas limites (*edge cases*) identifiés et corrigés :

- mise à l'échelle provoquant des décalages dans l'interface ;
- Le joueur peut se retrouver coincé dans les murs en cas de mauvaise construction de salles (obligation de mettre des murs d'une certaine façon) ;
- ennemis restant bloqués indéfiniment dans les coins extérieurs des murs (corrigé par un mécanisme de ligne de vue et de déblocage).

7 Conclusion

Ce rapport a présenté le développement de notre projet de jeu vidéo 2D de type Zelda, développé en Python avec le framework PyQt5.

7.1 Bilan des réalisations

Depuis le rapport intermédiaire, les fonctionnalités suivantes ont été implémentées :

- un système de PNJ et de dialogues animés lettre par lettre;
- un système de sauvegarde/chargement de parties en JSON;
- un système d'inventaire complet avec sérialisation;
- des objets interactifs (coffres, portes verrouillées, interrupteurs cristal, panneaux, points de sauvegarde);
- Plusieurs boss à patterns de combat (Tungtungsahur, Poseidon...);
- un bestiaire enrichi (Arakwa, Wizmount, Fly, Crouby, Polasu);
- de nouvelles armes (bombes, améliorations d'épée);
- un système de biomes avec chargement dynamique des tuiles;
- un monde de plus de 100 salles;
- un *flood fill* récursif pour l'optimisation du pathfinding;
- l'ensemble des ressources graphiques et sonores créées manuellement.
- une quête d'histoire se profile...

7.2 Apports personnels

Au-delà du cahier des charges initial, nous avons apporté plusieurs éléments qui dépassent le cadre d'un exercice académique :

- la construction d'un moteur de jeu 2D fonctionnel, évolutif et robuste à partir de zéro en Python avec un framework non prévu à cet effet (PyQt5);
- la création intégrale de toutes les ressources multimédia du jeu (pixel art, musiques, effets sonores);
- initiation au game design, avec la création de rooms intuitives et l'intention de guider le joueur au mieux sans qu'il soit perdu.

7.3 Perspectives et améliorations

Les pistes d'amélioration envisagées incluent :

- le développement du biome donjon (dernier biome du jeu);
- approfondissement et réflexion sur de nouvelles énigmes possibles grâce aux mécaniques implémentées (interrupteurs, murs cassables, blocs conditionnels...);
- l'ajout de secrets et de zones cachées exploitant le système de drapeaux;
- l'empaquetage du jeu en exécutable autonome (.exe);
- Prise en compte des retours des joueurs afin d'améliorer l'expérience utilisateur

Nous espérons finalement que vous prendrez autant de plaisir à jouer à notre jeu que nous avons eu de plaisir à le développer.



FIGURE 5 – Vue de deux salles du Donjon Final